

[Before you start](#)

[Download project starter files and deploy Azure services](#)

[Deploy the container app and configure secrets](#)

[Verify the deployment](#)

[Clean up resources](#)

[Troubleshooting](#)

# Deploy a containerized backend API to Azure Container Apps

In this exercise, you deploy a containerized backend API to Azure Container Apps. You use a managed identity to securely pull images from Azure Container Registry and configure secrets as environment variables.

Tasks performed in this exercise:

- Download the project starter files and deploy Azure services
- Deploy the container app with managed identity authentication
- Configure secrets and reference them from environment variables
- Verify the deployment by calling API endpoints and reviewing logs

This exercise takes approximately **30** minutes to complete.

**Important:** Azure Container Registry task runs are temporarily paused from Azure free credits. This exercise requires a Pay-As-You-Go, or another paid plan.

## Before you start

To complete the exercise, you need:

- An Azure subscription with the permissions to deploy the necessary Azure services. If you don't already have one, you can [sign up for one](#).
- [Visual Studio Code](#) on one of the [supported platforms](#).
- The latest version of the [Azure CLI](#).
- Optional: [Python 3.12](#) or greater.

## Download project starter files and deploy Azure services

In this section you download the project starter files and use a script to deploy the necessary services to your Azure subscription. The Azure Container Registry and Container Apps environment deployment takes a few minutes to complete.

1. Open a browser and enter the following URL to download the starter file. The file will be saved in your default download location.

Code

<https://github.com/MicrosoftLearning/mslearn-azure-ai/raw/main/downloads/python/aca-deploy-python.zip>
2. Copy, or move, the file to a location in your system where you want to work on the project. Then unzip the file into a folder.
3. Launch Visual Studio Code (VS Code) and select **File > Open Folder...** in the menu, then choose the folder containing the project files.
4. Open the *azdeploy.py* deployment script and change the two values at the top of the script to meet your needs, then save your changes. **Note:** Do not change anything else in the script.

Code

```
"<your-resource-group-name>" # Resource Group name
"<your-azure-region>" # Azure region for the resources
```

5. In the menu bar select **Terminal > New Terminal** to open a terminal window in VS Code.
6. Run the following command to login to your Azure account. Answer the prompts to select your Azure account and subscription for the exercise.

Code

```
az login
```

7. Run the following command to ensure you have the **containerapp** extension for Azure CLI.

|                                                                                       |      |
|---------------------------------------------------------------------------------------|------|
| Code                                                                                  | Copy |
| <pre>az extension add --name containerapp az extension add --name log-analytics</pre> |      |

8. Run the following commands to ensure your subscription has the necessary resource providers for the exercise.

|                                                                                                                                                                                       |      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| Code                                                                                                                                                                                  | Copy |
| <pre>az provider register --namespace Microsoft.App az provider register --namespace Microsoft.OperationalInsights az provider register --namespace Microsoft.ContainerRegistry</pre> |      |

## Create resources in Azure

In this section you run the deployment script to deploy the necessary services to your Azure subscription.

1. Make sure you are in the root directory of the project and run the following command in the terminal to launch the deployment script. The deployment script deploys ACR and creates the environment variable files needed for the exercise.

|                               |      |
|-------------------------------|------|
| Code                          | Copy |
| <pre>python azdeploy.py</pre> |      |

2. When the script is running, enter **1** to launch the **Create Azure Container Registry and build container image** option. This option creates the ACR service and uses ACR Tasks to build and push the image to the registry.
3. When the previous operation is finished, enter **2** to launch the **Create Container Apps environment** options. Creating the environment is necessary before deploying the container.

**Note:** A file containing environment variables is created after the Container Apps environment is created. You use these variables throughout the exercise.

4. When the previous operation is finished, enter **4** to exit the deployment script.
5. Run the appropriate command to load the environment variables into your terminal session from the file created in a previous step.

### Bash

|                        |      |
|------------------------|------|
| Code                   | Copy |
| <pre>source .env</pre> |      |

### PowerShell

|                        |      |
|------------------------|------|
| Code                   | Copy |
| <pre>. .\env.ps1</pre> |      |

**Note:** Keep the terminal open. If you close it and create a new terminal, you might need to run the command to create the environment variable again.

## Deploy the container app and configure secrets

In this section you deploy the API as a container app with external ingress. Because the image is in a private registry, you must configure registry authentication at create time so the first revision can pull the image. You then configure a secret and reference it from an environment variable. This pattern mirrors how AI apps store provider API keys

1. Create the container app with a system-assigned managed identity and configure registry authentication at create time. The **--registry-identity** flag tells Container Apps to use the app's managed identity to pull images from the specified registry. The CLI automatically assigns the **AcrPull** role when you use this flag with an Azure Container Registry.

Bash

|                                                                                                                                                                                                                                                                                                                                                                                                |      |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| Code                                                                                                                                                                                                                                                                                                                                                                                           | Copy |
| <pre>az containerapp create \<br/>  --name \$CONTAINER_APP_NAME \<br/>  --resource-group \$RESOURCE_GROUP \<br/>  --environment \$ACA_ENVIRONMENT \<br/>  --image "\$ACR_SERVER/\$CONTAINER_IMAGE" \<br/>  --ingress external \<br/>  --target-port \$TARGET_PORT \<br/>  --env-vars MODEL_NAME=\$MODEL_NAME \<br/>  --registry-server "\$ACR_SERVER" \<br/>  --registry-identity system</pre> |      |

PowerShell

|                                                                                                                                                                                                                                                                                                                                                                                                                                |      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| Code                                                                                                                                                                                                                                                                                                                                                                                                                           | Copy |
| <pre>az containerapp create \<br/>  --name \$env:CONTAINER_APP_NAME \<br/>  --resource-group \$env:RESOURCE_GROUP \<br/>  --environment \$env:ACA_ENVIRONMENT \<br/>  --image "\$env:ACR_SERVER/\$env:CONTAINER_IMAGE" \<br/>  --ingress external \<br/>  --target-port \$env:TARGET_PORT \<br/>  --env-vars MODEL_NAME=\$env:MODEL_NAME \<br/>  --registry-server "\$env:ACR_SERVER" \<br/>  --registry-identity system</pre> |      |

2. Create a secret and reference it from an environment variable.

Bash

|                                                                                                                                             |      |
|---------------------------------------------------------------------------------------------------------------------------------------------|------|
| Code                                                                                                                                        | Copy |
| <pre>az containerapp secret set -n \$CONTAINER_APP_NAME -g \$RESOURCE_GROUP \<br/>  --secrets embeddings-api-key=\$EMBEDDINGS_API_KEY</pre> |      |

PowerShell

|                                                                                                                                                         |      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| Code                                                                                                                                                    | Copy |
| <pre>az containerapp secret set -n \$env:CONTAINER_APP_NAME -g \$env:RESOURCE_GROUP \<br/>  --secrets embeddings-api-key=\$env:EMBEDDINGS_API_KEY</pre> |      |

3. Reference the secret from an environment variable. This command creates a new revision, which restarts the app so the secret change takes effect.

Bash

|                                                                                                                                                      |      |
|------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| Code                                                                                                                                                 | Copy |
| <pre>az containerapp update -n \$CONTAINER_APP_NAME -g \$RESOURCE_GROUP \<br/>  --set-env-vars EMBEDDINGS_API_KEY=secretref:embeddings-api-key</pre> |      |

PowerShell

|                                                                                                                                                              |      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| Code                                                                                                                                                         | Copy |
| <pre>az containerapp update -n \$env:CONTAINER_APP_NAME -g \$env:RESOURCE_GROUP \<br/>  --set-env-vars EMBEDDINGS_API_KEY=secretref:embeddings-api-key</pre> |      |

4. Run the following command to list the revisions to confirm a new revision was created.

Bash

|                                                                                               |      |
|-----------------------------------------------------------------------------------------------|------|
| Code                                                                                          | Copy |
| <pre>az containerapp revision list -n \$CONTAINER_APP_NAME -g \$RESOURCE_GROUP -o table</pre> |      |

PowerShell

|      |      |
|------|------|
| Code | Copy |
|------|------|

```
az containerapp revision list -n $env:CONTAINER_APP_NAME -g $env:RESOURCE_GROUP -o table
```

The revision name ends with a suffix like `--000002`, indicating this is the second revision. Container Apps creates a new revision whenever you change environment variables or secrets, which restarts the app with the updated configuration. Old inactive revisions may be pruned over time.

## Verify the deployment

You should validate that the app starts and that ingress works. You also use logs to confirm the app is behaving as expected.

1. Run the following command to retrieve the app FQDN and store the result to a variable.

### Bash

|                                                                                                                                                              |      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| Code                                                                                                                                                         | Copy |
| <pre>FQDN=\$(az containerapp show -n \$CONTAINER_APP_NAME -g \$RESOURCE_GROUP \   --query properties.configuration.ingress.fqdn -o tsv)  echo "\$FQDN"</pre> |      |

### PowerShell

|                                                                                                                                                                            |      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| Code                                                                                                                                                                       | Copy |
| <pre>\$FQDN = az containerapp show -n \$env:CONTAINER_APP_NAME -g \$env:RESOURCE_GROUP `   --query properties.configuration.ingress.fqdn -o tsv  Write-Output \$FQDN</pre> |      |

2. Run the following command to call the health endpoint. The command should return `{"status": "healthy"}`.

### Bash

|                                            |      |
|--------------------------------------------|------|
| Code                                       | Copy |
| <pre>curl -s "https://\$FQDN/health"</pre> |      |

### PowerShell

|                                                           |      |
|-----------------------------------------------------------|------|
| Code                                                      | Copy |
| <pre>Invoke-RestMethod -Uri "https://\$FQDN/health"</pre> |      |

3. Run the following command to verify the secret is configured by calling the root endpoint. The endpoint returns JSON containing app information including the configured model name and whether the API key secret is configured.

### Bash

|                                      |      |
|--------------------------------------|------|
| Code                                 | Copy |
| <pre>curl -s "https://\$FQDN/"</pre> |      |

### PowerShell

|                                                     |      |
|-----------------------------------------------------|------|
| Code                                                | Copy |
| <pre>Invoke-RestMethod -Uri "https://\$FQDN/"</pre> |      |

4. Run the following command to test the document processing endpoint. The command sends the `document.txt` file to the endpoint. The operation returns JSON with mock data analysis information.

### Bash

|      |      |
|------|------|
| Code | Copy |
|------|------|

```
curl -s -X POST "https://$FQDN/process" \  
  -H "Content-Type: text/plain" \  
  -d @document.txt
```

PowerShell

|                                                                                                                                             |      |
|---------------------------------------------------------------------------------------------------------------------------------------------|------|
| Code                                                                                                                                        | Copy |
| Invoke-RestMethod -Uri "https://\$FQDN/process" \<br>-Method Post \<br>-ContentType "text/plain" \<br>-Body (Get-Content -Raw document.txt) |      |

5. Run the following command to review logs for startup and runtime signals. This command shows recent console output only. For historical logs and advanced troubleshooting, logs persist in the Log Analytics workspace associated with your Container Apps environment.

Bash

|                                                                       |      |
|-----------------------------------------------------------------------|------|
| Code                                                                  | Copy |
| az containerapp logs show -n \$CONTAINER_APP_NAME -g \$RESOURCE_GROUP |      |

Powershell

|                                                                               |      |
|-------------------------------------------------------------------------------|------|
| Code                                                                          | Copy |
| az containerapp logs show -n \$env:CONTAINER_APP_NAME -g \$env:RESOURCE_GROUP |      |

Look for **gunicorn** startup messages showing workers spawned and listening on port 8000. You should also see HTTP request logs from your curl commands (GET /health, POST /process, etc.).

## Clean up resources

Now that you finished the exercise, you should delete the cloud resources you created to avoid unnecessary resource usage.

1. Run the following command in the VS Code terminal to delete the resource group, and all resources in the group. Replace **<rg-name>** with the name you choose earlier in the exercise. The command will launch a background task in Azure to delete the resource group.

|                                                  |      |
|--------------------------------------------------|------|
| Code                                             | Copy |
| az group delete --name <rg-name> --no-wait --yes |      |

**CAUTION:** Deleting a resource group deletes all resources contained within it. If you chose an existing resource group for this exercise, any existing resources outside the scope of this exercise will also be deleted.

## Troubleshooting

If you encounter issues while completing this exercise, try the following troubleshooting steps:

Check deployment status with the script

- Run the deployment script and select option **3** to check the status of your ACR and Container Apps environment. This verifies the base infrastructure is deployed and the container image exists.

Verify Azure authentication and environment variables

- Run **az account show** to confirm you're logged in to the correct Azure subscription.
- Verify your environment variables are set by running **echo \$ACR\_NAME** (Bash) or **\$env:ACR\_NAME** (PowerShell).
- If variables are empty, re-run **source .env** (Bash) or **..env.ps1** (PowerShell).

Verify ACR deployment

- Navigate to the [Azure portal](#) and locate your resource group.
- Confirm that the Azure Container Registry exists and shows a **Provisioning State** of **Succeeded**.
- Run **az acr list --output table** to verify your registry is accessible.



## Troubleshoot build failures

- The deployment script suppresses verbose **az acr build** output. To troubleshoot failures, check the status and logs of the most recent ACR Task run.
- Verify you're running the deployment script from the project root directory (where the *api* folder is located).
- List recent ACR Task runs:
  - **Bash:** `az acr task list-runs --registry $ACR_NAME --output table`
  - **PowerShell:** `az acr task list-runs --registry $env:ACR_NAME --output table`
- View logs for a specific run (replace **<run-id>** with a value from the previous command):
  - **Bash:** `az acr task logs --registry $ACR_NAME --run-id <run-id>`
  - **PowerShell:** `az acr task logs --registry $env:ACR_NAME --run-id <run-id>`

## Troubleshoot container pull failures (ImagePullBackOff / unauthorized / 403)

- Confirm the container app has a system-assigned managed identity enabled:
  - **Bash:** `az containerapp identity show -n $CONTAINER_APP_NAME -g $RESOURCE_GROUP`
  - **PowerShell:** `az containerapp identity show -n $env:CONTAINER_APP_NAME -g $env:RESOURCE_GROUP`
- Confirm the container app has the **AcrPull** role assignment scoped to the registry. Role assignments can take a minute or two to propagate after creation.

## Troubleshoot container startup and application errors

- Stream container logs to diagnose startup issues:
  - **Bash:** `az containerapp logs show -n $CONTAINER_APP_NAME -g $RESOURCE_GROUP --follow`
  - **PowerShell:** `az containerapp logs show -n $env:CONTAINER_APP_NAME -g $env:RESOURCE_GROUP --follow`
- If the app returns a 502/503 shortly after deployment, wait a minute and try again. The first start can take longer while Container Apps pulls and starts the container.
- Check revision status for provisioning errors:
  - **Bash:** `az containerapp revision list -n $CONTAINER_APP_NAME -g $RESOURCE_GROUP -o table`
  - **PowerShell:** `az containerapp revision list -n $env:CONTAINER_APP_NAME -g $env:RESOURCE_GROUP -o table`

## Troubleshoot secret configuration

- Verify the secret was created:
  - **Bash:** `az containerapp secret list -n $CONTAINER_APP_NAME -g $RESOURCE_GROUP -o table`
  - **PowerShell:** `az containerapp secret list -n $env:CONTAINER_APP_NAME -g $env:RESOURCE_GROUP -o table`
- Confirm the environment variable references the secret correctly by calling the root endpoint (*/*), which shows whether the API key is configured.